Practical

Full Name:	**DIPAK RAJENDRA POTE**
DIV:	**IT-G**
Roll No.:	**31**
PRN:	**12520009**
Subject:	**Computer Networks**
Date of Submission:	**27/08/2026**

Aim:

Write a program to demonstrate subnetting and find the subnet masks.

Theory:

Subnetting

Subnetting is the process of dividing a single IP network into smaller sub-networks (subnets). It allows efficient utilization of IP address space and improves network management and security. Subnetting is achieved by borrowing bits from the host portion of an IP address to extend the network portion, thereby creating additional subnets.

Subnet Mask

A subnet mask is a 32-bit number that distinguishes the network portion of an IP address from the host portion. It works in conjunction with the IP address to define the network and host parts. For example, the default subnet mask for a Class C network is 255.255.255.0, which means the first 24 bits identify the network and the last 8 bits identify hosts.

CIDR Notation

Classless Inter-Domain Routing (CIDR) notation expresses a subnet mask as a prefix length after a slash (e.g., /24 means the first 24 bits are the network part). When subnetting, bits are borrowed from the host portion, increasing the prefix length and creating more subnets with fewer hosts each.

Subnetting Calculation

When subnetting is required, the number of bits to borrow is determined by finding the smallest power of 2 greater than or equal to the required number of subnets. The block size (the number of addresses in each subnet) is 2^(host bits remaining). Each subnet has:

- Network Address: The first address of the subnet (not assignable to hosts)
- First Usable IP: Network address + 1
- Last Usable IP: Broadcast address - 1
- Broadcast Address: The last address of the subnet (used to address all hosts in subnet)

Classes and Default CIDRs

- Class A (1–126): Default /8, Subnet Mask 255.0.0.0
- Class B (128–191): Default /16, Subnet Mask 255.255.0.0
- Class C (192–223): Default /24, Subnet Mask 255.255.255.0

Algorithm:

Step 1: Read the IP address as four integers a, b, c, d and the number of required subnets.

Step 2: Validate each octet: if any value is outside 0–255, print "Invalid IP Address" and stop.

Step 3: Determine the IP class and default CIDR from the first octet:

- 1–126 → Class A (default /8)
- 128–191 → Class B (default /16)
- 192–223 → Class C (default /24)

Step 4: Calculate bits to borrow: find the smallest value of bits such that 2^bits ≥ number of subnets.

Step 5: Compute the new CIDR = default CIDR + bits borrowed.

Step 6: Calculate the new subnet mask from the new CIDR.

Step 7: Compute block size = 2^(32 – new CIDR).

Step 8: For each subnet (i = 0 to 2^bits – 1), compute:

- Network IP = base_ip + i × block
- First Usable IP = Network IP + 1
- Broadcast IP = Network IP + block – 1
- Last Usable IP = Broadcast IP – 1

Step 9: Determine type: Private or Public using RFC 1918 ranges.

Step 10: Print all subnet information in tabular format.

Step 11: Stop.

Code:

```
#include <stdio.h>

int main() {
    int a, b, c, d;

    printf("Enter IP Address (e.g. 192.168.1.10): ");
    if (scanf("%d.%d.%d.%d", &a, &b, &c, &d) != 4) {
        printf("Invalid IP Address format\n");
        return 0;
    }

    if (a < 0 || a > 255 || b < 0 || b > 255 || c < 0 || c > 255 || d < 0 || d >
255) {
        printf("Invalid IP Address\n");
        return 0;
    }

    printf("\nHow many Subnets do you want to create? ");
    int subnets;
    scanf("%d", &subnets);

    if (subnets <= 0) {
        printf("Number of subnets must be greater than 0.\n");
        return 0;
    }

    int bits_needed = 0;
    int total_subnets = 1;
    while (total_subnets < subnets) {
        total_subnets *= 2;
        bits_needed++;
    }

    printf("\n");

    int default_mask_octets = 0;
    char class_name = ' ';

    if (a >= 1 && a <= 126) {
```

```
        class_name = 'A';
        default_mask_octets = 1; // 255.0.0.0
    }
    else if (a >= 128 && a <= 191) {
        class_name = 'B';
        default_mask_octets = 2; // 255.255.0.0
    }
    else if (a >= 192 && a <= 223) {
        class_name = 'C';
        default_mask_octets = 3; // 255.255.255.0
    }
    else if (a >= 224 && a <= 239) {
        printf("Class Name        : D (Multicast)\n");
        return 0;
    }
    else if (a >= 240 && a <= 255) {
        printf("Class Name        : E (Experimental)\n");
        return 0;
    }
    else if (a == 127) {
        printf("Class Name        : Loopback\n");
        return 0;
    }
    else {
        printf("Invalid IP Address\n");
        return 0;
    }

    if (bits_needed > 8) {
        printf("Cannot borrow more than 8 bits in a single octet for simple
subnetting.\n");
        return 0;
    }

    // IP Type
    if ((a == 10) || (a == 172 && b >= 16 && b <= 31) || (a == 192 && b == 168))
        printf("Type              : Private\n");
    else
        printf("Type              : Public\n");

    printf("Class Name        : %c\n", class_name);

    // 3. Block Size Calculation
    int block_size = 256 / total_subnets;
    int custom_octet_value = 256 - block_size;

    // 4. Subnet Mask Construction
    int mask_a = 255, mask_b = 0, mask_c = 0, mask_d = 0;

    if (class_name == 'C') {
        mask_b = 255;
        mask_c = 255;
        mask_d = custom_octet_value;
    }
    else if (class_name == 'B') {
        mask_b = 255;
        mask_c = custom_octet_value;
        mask_d = 0;
```

```
    }
    else if (class_name == 'A') {
        mask_b = custom_octet_value;
        mask_c = 0;
        mask_d = 0;
    }

    int default_cidr = default_mask_octets * 8;
    int new_cidr = default_cidr + bits_needed;

    printf("Default CIDR      : /%d\n", default_cidr);
    printf("New Subnet Mask   : %d.%d.%d.%d (/%d)\n", mask_a, mask_b, mask_c,
mask_d, new_cidr);
    printf("Bits Borrowed     : %d\n", bits_needed);
    printf("Block Size        : %d\n", block_size);

    // 5. Formatted Output Table
    printf("\
n-------------------------------------------------------------------------------
-------------------\n");
    printf("%-8s %-20s %-20s %-20s %-20s\n", "Subnet", "Network IP", "First
Usable IP", "Last Usable IP", "Broadcast IP");

printf("-------------------------------------------------------------------------
-------------------------\n");

    for (int i = 0; i < total_subnets; i++) {
        int start_val = i * block_size;
        int end_val = start_val + block_size - 1;

        char net_ip[30], first_ip[30], last_ip[30], bcast_ip[30];

        if (class_name == 'C') {
            snprintf(net_ip, sizeof(net_ip), "%d.%d.%d.%d", a, b, c, start_val);
            snprintf(first_ip, sizeof(first_ip), "%d.%d.%d.%d", a, b, c,
start_val + 1);
            snprintf(last_ip, sizeof(last_ip), "%d.%d.%d.%d", a, b, c, end_val -
1);
            snprintf(bcast_ip, sizeof(bcast_ip), "%d.%d.%d.%d", a, b, c,
end_val);
        }
        else if (class_name == 'B') {
            snprintf(net_ip, sizeof(net_ip), "%d.%d.%d.0", a, b, start_val);
            snprintf(first_ip, sizeof(first_ip), "%d.%d.%d.1", a, b, start_val);
            snprintf(last_ip, sizeof(last_ip), "%d.%d.%d.254", a, b, end_val);
            snprintf(bcast_ip, sizeof(bcast_ip), "%d.%d.%d.255", a, b, end_val);
        }
        else if (class_name == 'A') {
            snprintf(net_ip, sizeof(net_ip), "%d.%d.0.0", a, start_val);
            snprintf(first_ip, sizeof(first_ip), "%d.%d.0.1", a, start_val);
            snprintf(last_ip, sizeof(last_ip), "%d.%d.255.254", a, end_val);
            snprintf(bcast_ip, sizeof(bcast_ip), "%d.%d.255.255", a, end_val);
        }

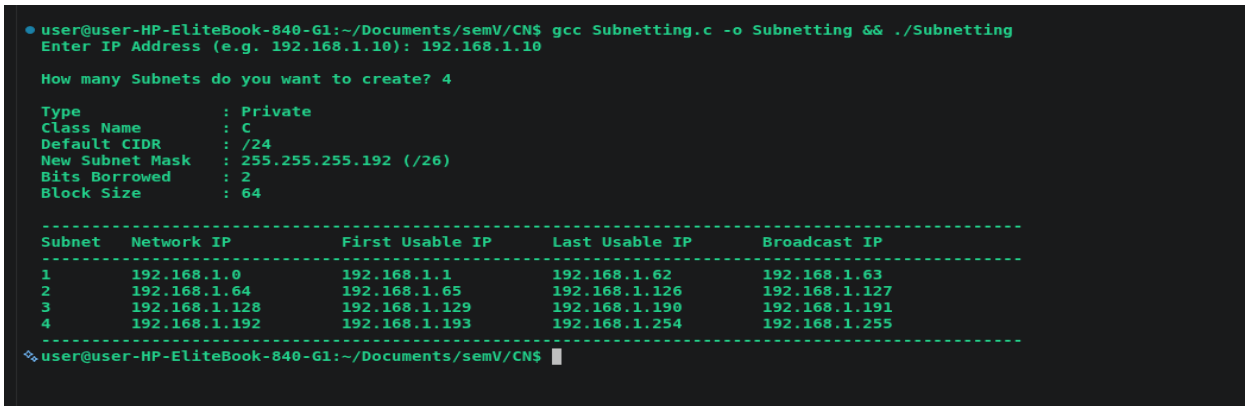
        printf("%-8d %-20s %-20s %-20s %-20s\n", i + 1, net_ip, first_ip,
last_ip, bcast_ip);
    }
```

```
    printf("-----------------------------------------------------------------------------
    -------------------------\n");

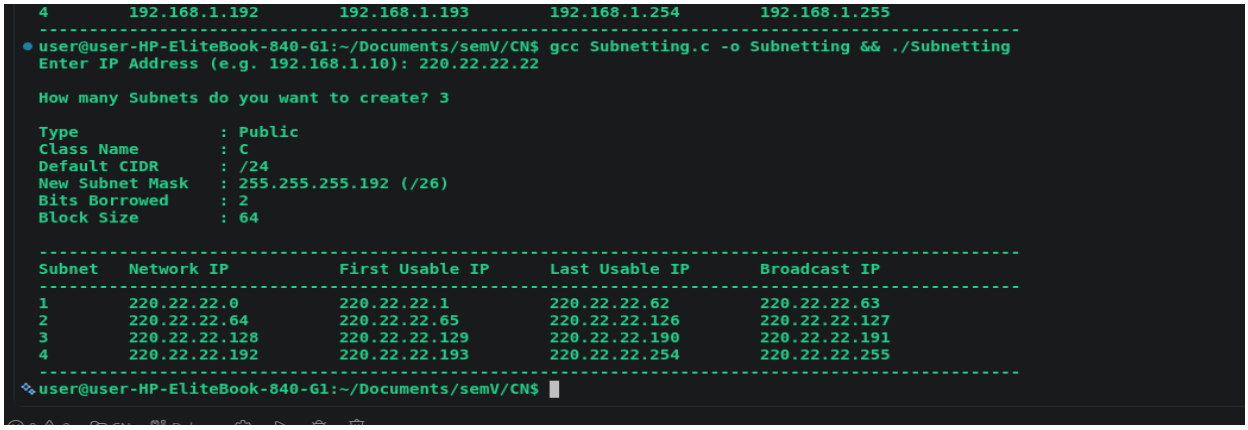
        return 0;
    }
```

Output:

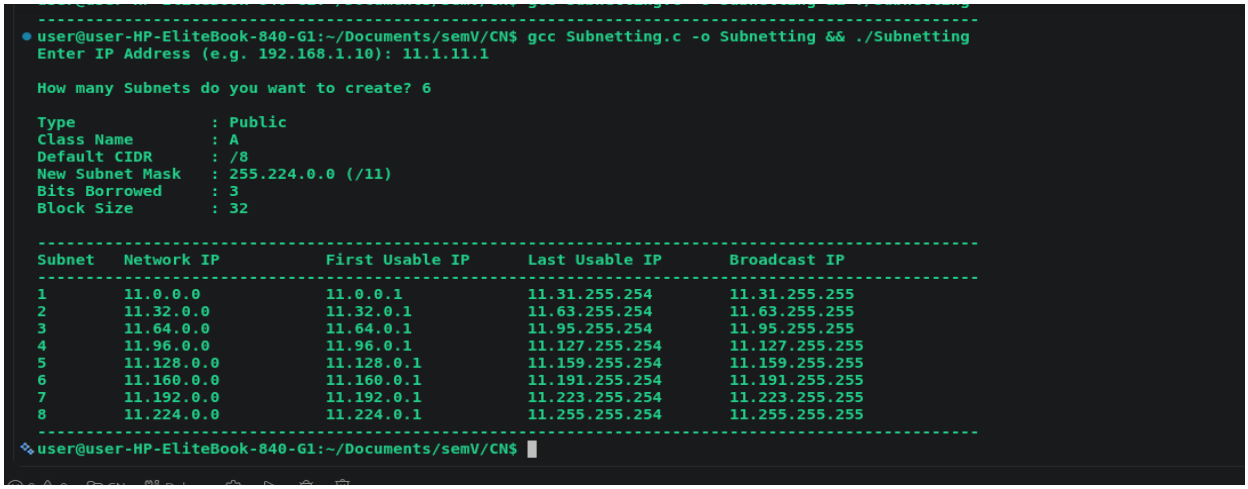
Test Case 1: IP = 192.168.1.10, Subnets = 4 (Private Class C)



Test Case 2: IP = 220.22.22.22, Subnets = 3 (Public Class C)



Test Case 3: IP = 11.1.11.1, Subnets = 6 (Public Class A)



Conclusion:

The program successfully demonstrates subnetting by reading an IPv4 address and the required number of subnets, then calculating and displaying all relevant subnet information. For Test Case 1 (192.168.1.10, 4 subnets), the program correctly identified a Private Class C address with default /24 CIDR, borrowed 2 bits to produce a /26 subnet mask (255.255.255.192) and a block size of 64, generating four subnets with proper network, usable host, and broadcast addresses. For Test Case 2 (220.22.22.22, 3 subnets), a Public Class C address was similarly subnetted into four /26 subnets (since 2 bits must be borrowed to accommodate at least 3 subnets). For Test Case 3 (11.1.11.1, 6 subnets), a Public Class A address with default /8 CIDR was subnetted using 3 borrowed bits to produce 8 subnets with a /11 mask (255.224.0.0) and a block size of 2,097,152 addresses each. Thus, the aim of the practical is fulfilled and the subnetting algorithm is verified across multiple IP classes and subnet requirements.